

Heart Rate Monitoring and Emergency Alert System Using Heart Rate Sensor

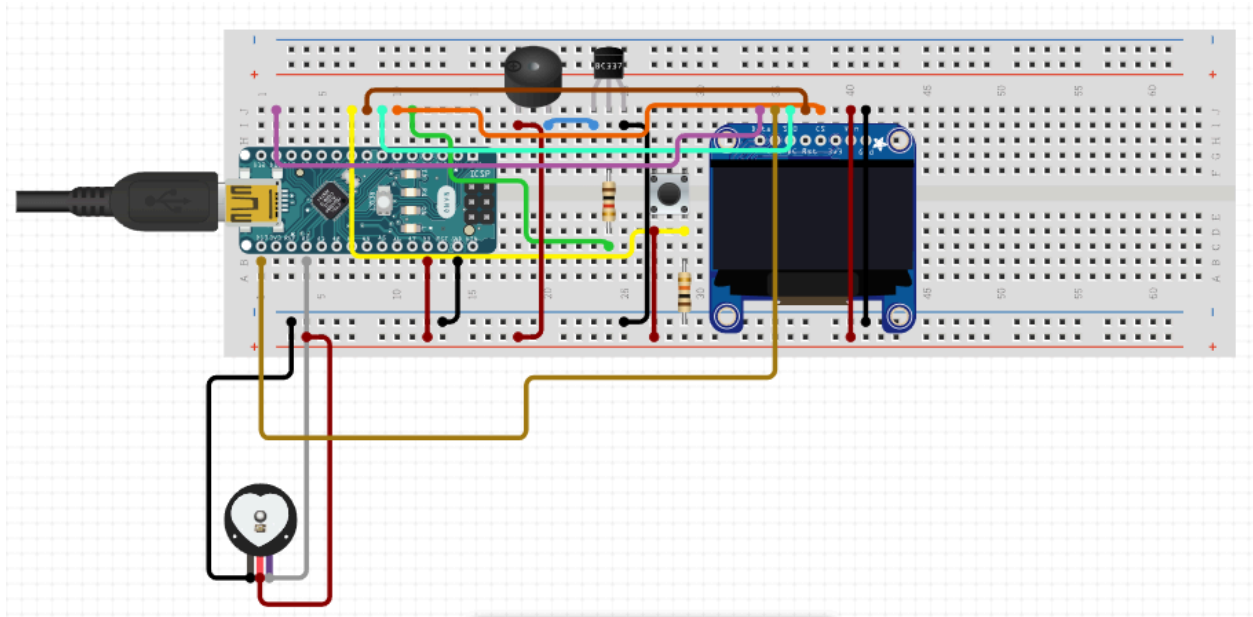
Name : Deepraj Naik

Roll No : 2519

● Components Required

Sr.No.	Component	Quantity	Description	Use
1.	Arduino Nano	1	It is a small microcontroller board.	It is used to control the whole system and process the sensor data.
2.	BreadBoard	1	It is a board used to connect components without soldering.	It is used to make the circuit connections.
3.	Buzzer	1	It is a device that produces a sound.	It is used to give an alert when an abnormal heart rate is detected.
4.	Pushbutton Switch	1	It is a button used to give input to the Arduino.	It is used to start or control the monitoring system.
5.	OLED Display	1	It is a small display used to show information.	It is used to display the heart rate (BPM) and system status.
6.	Heart Rate Sensor	1	It is a sensor that detects the heartbeat and measures the heart rate.	It is used to measure the heart rate of a person.

● Circuit Diagram



- Program Code

heartensor.ino

```
#define USE_ARDUINO_INTERRUPTS true

#include <PulseSensorPlayground.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// ===== OLED =====

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// ===== HEART SENSOR =====

#define PULSE_PIN A0
#define THRESHOLD 520

PulseSensorPlayground pulseSensor;

// ===== BUZZER =====

#define BUZZER_PIN 5
```

```

// ===== BUTTON =====

#define BUTTON_PIN 2

// ===== LIMITS =====

#define HIGH_BPM 100

#define MIN_VALID_BPM 40
#define MAX_VALID_BPM 180

int BPM = 0;
int lastValidBPM = 0;

unsigned long lastBeatTime = 0;

// Buzzer state
bool buzzerMuted = false;

// Button state
bool lastButtonState = HIGH;

void setup() {

    Serial.begin(9600);

    // Buzzer
    pinMode(BUZZER_PIN, OUTPUT);
    noTone(BUZZER_PIN);

    // Button
    pinMode(BUTTON_PIN, INPUT_PULLUP);

    // ===== OLED =====

    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {

        Serial.println("OLED not found!");

        while (1);
    }

    display.clearDisplay();

```

```

display.setTextColor(SSD1306_WHITE);

display.setTextSize(2);
display.setCursor(20, 5);
display.println("HEART");

display.setCursor(30, 30);
display.println("RATE");

display.display();

delay(2000);

// ===== PULSE SENSOR =====

pulseSensor.analogInput(PULSE_PIN);
pulseSensor.setThreshold(THRESHOLD);

pulseSensor.begin();

// ===== READY SCREEN =====

display.clearDisplay();

display.setTextSize(1);
display.setCursor(25, 20);
display.println("Place finger");

display.setCursor(30, 35);
display.println("on sensor");

display.display();

delay(2000);
}

void loop() {

// =====
// BUTTON CHECK
// =====

bool buttonState = digitalRead(BUTTON_PIN);

```

```

// Button pressed
if (lastButtonState == HIGH && buttonState == LOW) {

    // Silence buzzer
    noTone(BUZZER_PIN);

    buzzerMuted = true;

    Serial.println("BUZZER MUTED");

    delay(200);
}

lastButtonState = buttonState;

// =====
// HEARTBEAT DETECTION
// =====

if (pulseSensor.sawStartOfBeat()) {

    BPM = pulseSensor.getBeatsPerMinute();

    // Accept only realistic readings
    if (BPM >= MIN_VALID_BPM && BPM <= MAX_VALID_BPM) {

        lastValidBPM = BPM;
        lastBeatTime = millis();

        // =====
        // SEND BPM TO PYTHON
        // =====

        Serial.println(lastValidBPM);

        // =====
        // HIGH HEART RATE
        // =====

        if (lastValidBPM > HIGH_BPM) {

            // Only sound if not muted
            if (!buzzerMuted) {

```

```

    tone(BUZZER_PIN, 2000);
}

} else {

    // Normal heart rate
    noTone(BUZZER_PIN);

    // Reset mute when heart rate becomes normal
    buzzerMuted = false;
}

// =====
// OLED
// =====

display.clearDisplay();

display.setTextColor(SSD1306_WHITE);

// Title
display.setTextSize(1);
display.setCursor(35, 0);
display.println("HEART RATE");

// Heart
display.setTextSize(2);
display.setCursor(5, 20);
display.print("<3");

// BPM
display.setCursor(45, 20);
display.print(lastValidBPM);

display.setTextSize(1);
display.setCursor(95, 27);
display.println("BPM");

// Status
if (lastValidBPM > HIGH_BPM) {

    display.setCursor(10, 50);

    if (buzzerMuted) {

```

```

        display.println("HIGH - MUTED");
    } else {
        display.println("HIGH HEART RATE!");
    }

} else {

    display.setCursor(40, 50);
    display.println("NORMAL");
}

display.display();
}
}

// =====
// NO FINGER DETECTED
// =====

if (millis() - lastBeatTime > 5000) {

    noTone(BUZZER_PIN);

    buzzerMuted = false;

    display.clearDisplay();

    display.setTextColor(SSD1306_WHITE);

    display.setTextSize(1);

    display.setCursor(35, 5);
    display.println("HEART RATE");

    display.setCursor(20, 25);
    display.println("Place finger");

    display.setCursor(30, 40);
    display.println("on sensor");

    display.display();
}

delay(20);

```

```

}
heart_rate_graph.py

import serial
import time
import matplotlib.pyplot as plt
from collections import deque

# =====
# SETTINGS
# =====

SERIAL_PORT = "COM4"    # CHANGE THIS
BAUD_RATE = 9600

MAX_POINTS = 60        # Show last 60 readings

HIGH_BPM = 100
LOW_BPM = 40

# =====
# CONNECT TO ARDUINO
# =====

try:
    arduino = serial.Serial(
        SERIAL_PORT,
        BAUD_RATE,
        timeout=1
    )

    time.sleep(2)

    print("Connected to Arduino!")
    print("Starting live heart-rate graph...")

except serial.SerialException:
    print("Could not connect to Arduino.")
    print("Check the COM port.")
    print("Also make sure Arduino Serial Monitor is CLOSED.")
    exit()

# =====
# DATA STORAGE

```

```

# =====

bpm_data = deque(maxlen=MAX_POINTS)
time_data = deque(maxlen=MAX_POINTS)

start_time = time.time()

# =====
# CREATE GRAPH
# =====

plt.ion()

fig, ax = plt.subplots()

line, = ax.plot([], [], marker="o")

ax.set_title("Real-Time Heart Rate Monitor")
ax.set_xlabel("Time (seconds)")
ax.set_ylabel("Heart Rate (BPM)")

ax.set_ylim(30, 180)
ax.grid(True)

# =====
# LIVE DATA LOOP
# =====

try:

    while True:

        if arduino.in_waiting > 0:

            data = arduino.readline().decode(
                "utf-8",
                errors="ignore"
            ).strip()

            if data:

                try:

                    bpm = int(data)

```

```

# Ignore impossible values
if LOW_BPM <= bpm <= 180:

    current_time = time.time() - start_time

    bpm_data.append(bpm)
    time_data.append(current_time)

# Print BPM
if bpm > HIGH_BPM:

    status = "HIGH"

else:

    status = "NORMAL"

print(
    f"BPM: {bpm} | Status: {status}"
)

# =====
# UPDATE GRAPH
# =====

line.set_xdata(time_data)
line.set_ydata(bpm_data)

# Automatically adjust X axis
if len(time_data) > 1:

    ax.set_xlim(
        time_data[0],
        time_data[-1] + 1
    )

fig.canvas.draw()
fig.canvas.flush_events()

except ValueError:
    pass

plt.pause(0.01)

```

```
except KeyboardInterrupt:
```

```
    print("\nMonitoring stopped.")
```

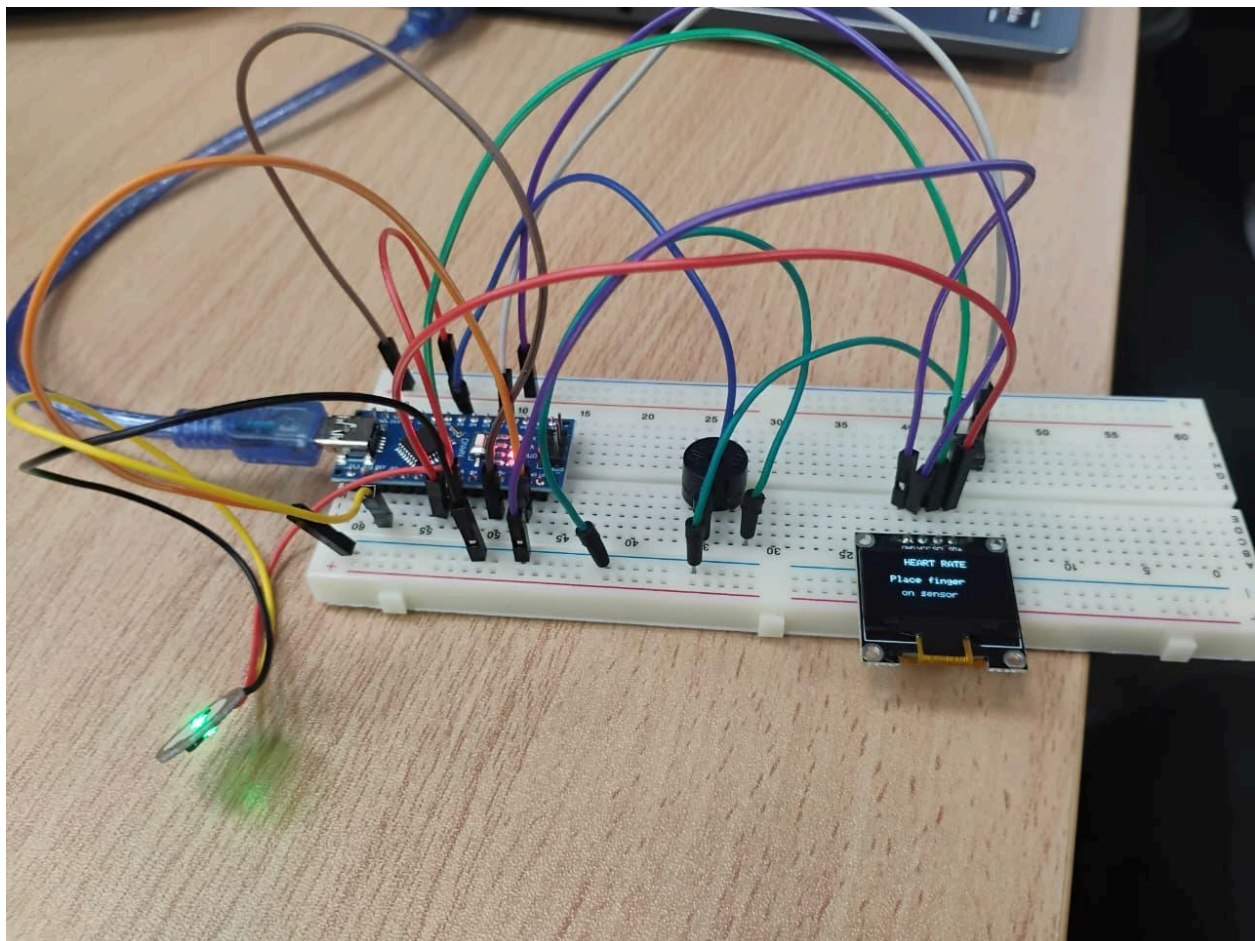
```
finally:
```

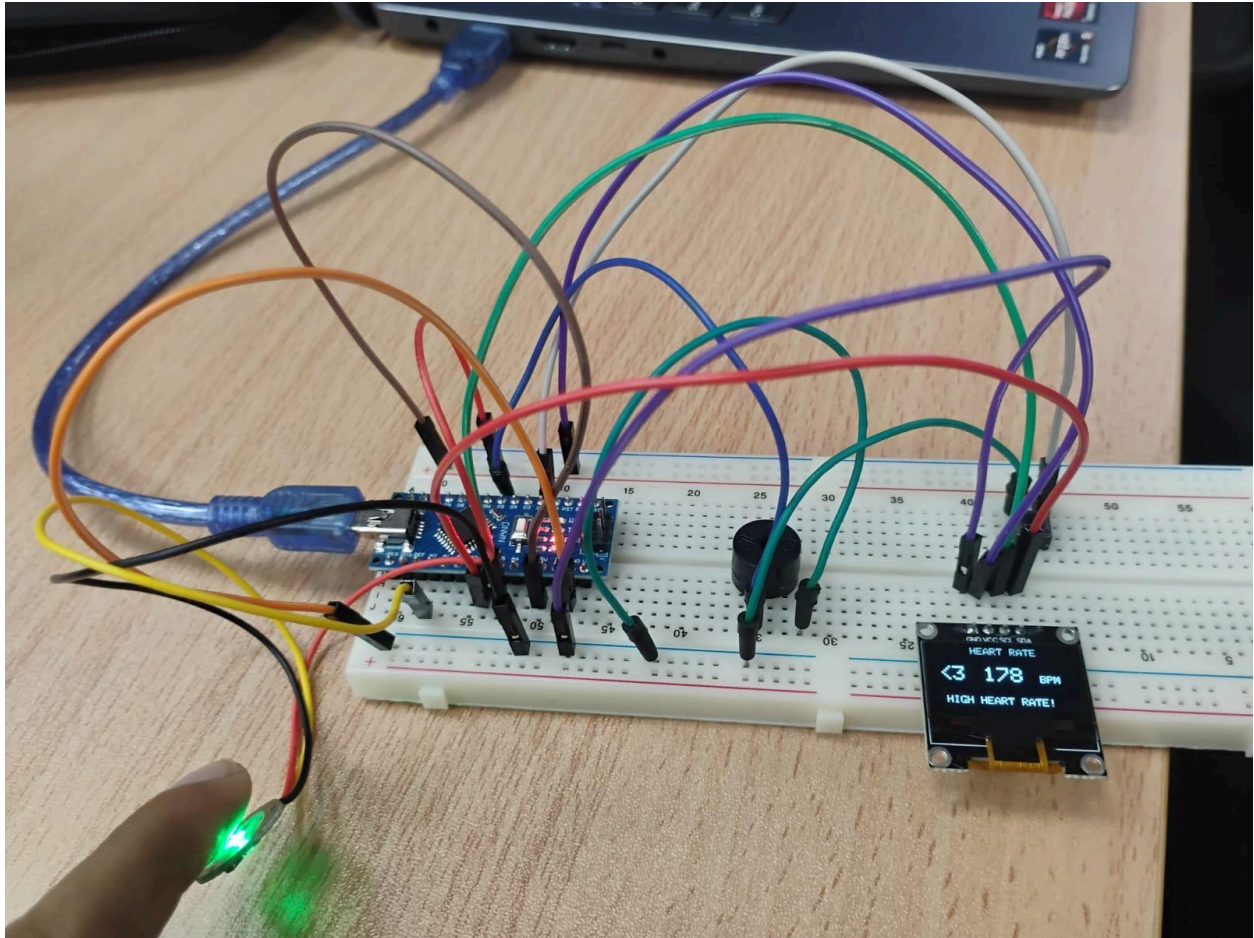
```
    arduino.close()
```

```
    plt.close()
```

```
    print("Arduino connection closed.")
```

- **Photograph of the Experiment**





- **Conclusion**

In this experiment, we learned how to make a simple heart rate monitoring system using Arduino Nano and a heart rate sensor. How to measure the heart rate and display it on the OLED display. Used a buzzer to give an alert when the heart rate is abnormal. This experiment helped us understand how sensors can be used for health monitoring and emergency alerts.